

Enterprise RAG

Modules 6–9 · Foundations → Vector Search → End-to-End →
Governance

What we'll in next 2 days

MODULE 6

Enterprise RAG Foundations

Architecture, retrieval workflow, chunking, metadata & preprocessing.

MODULE 7

Embeddings & Vector Search

Embeddings, vector DBs, reranking, hybrid & metadata filtering.

MODULE 8

End-to-End Enterprise RAG

Context assembly, multimodal, graph & audio RAG, scaling limits.

MODULE 9

Evaluation, Reliability & Governance

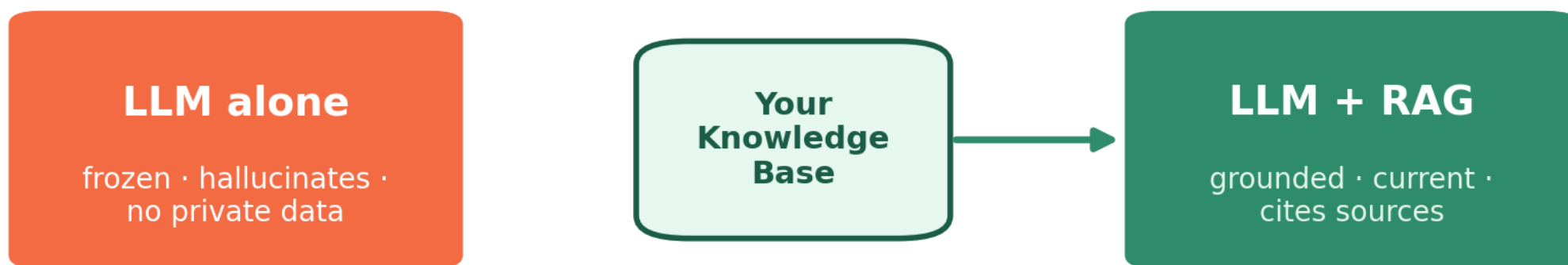
Groundedness, RAGAS, golden sets, responsible-AI controls.

MODULE 6 Enterprise RAG Foundations

RAG architecture • retrieval workflow • chunking • metadata design •
document preprocessing

Why RAG? Giving the model memory on demand

LLMs are powerful but frozen, unaware of your private data, and prone to hallucination. RAG fixes all three.



Retrieval gives the model memory on demand

Key takeaway: Retrieve first, then generate — answers become grounded, current and citable.

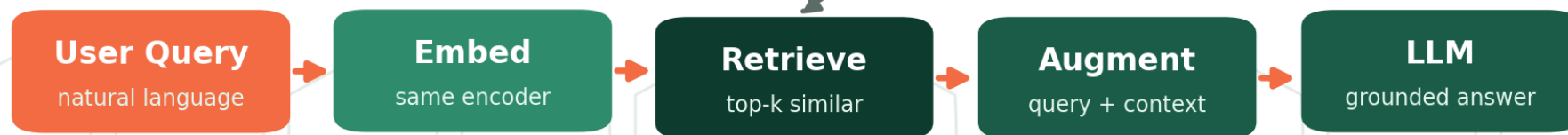
RAG architecture: two pipelines

Indexing happens offline; retrieval + generation happen live on every query.

INDEXING (offline)



QUERY (online)



similarity
search

Document ingestion & preprocessing

- **Loaders** — PDF, HTML, DOCX, CSV, databases, wikis, tickets — each needs its own parser.
- **Clean** — strip boilerplate, headers/footers, navigation; normalise whitespace & encoding.
- **Structure** — preserve headings, tables and lists so meaning survives chunking.
- **Enrich** — attach metadata (source, date, department, doc-type, access level).
- **De-duplicate** — near-identical policy versions bloat the index and confuse retrieval.

PDF / scans

OCR + layout models

Tables

flatten → CSV / SQL

Metadata

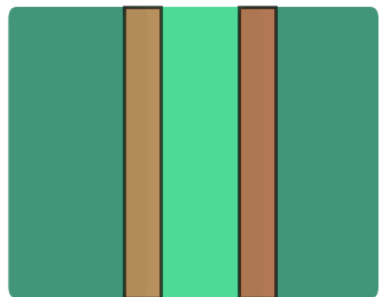
the governance layer

Key takeaway: Garbage in, garbage retrieved — preprocessing is where enterprise RAG is won or lost.

Chunking strategies

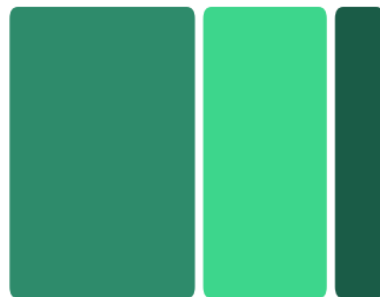
Chunking splits documents so each piece fits the embedder and carries one coherent idea.

Fixed-size + overlap



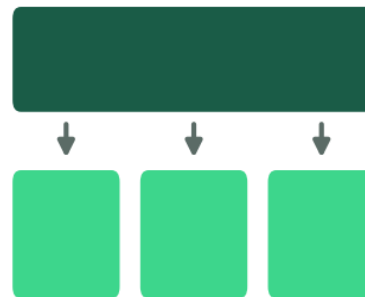
overlap keeps context

Semantic



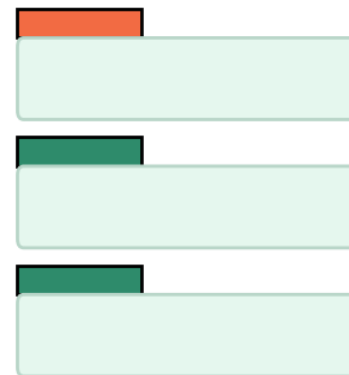
split when meaning shifts

Recursive



split only if too large

Document-based



respects headings/sections

Sliding window



window slides w/ shared text

Key takeaway: No universal winner — match the strategy to document structure and query style.

Choosing chunk size: the trade-off

- **Larger chunks** — more context per hit, but dilute relevance and cost more tokens.
- **Smaller chunks** — sharp relevance, but ideas get split across pieces.
- **Overlap** — 10–20% keeps sentences from being cut mid-thought.
- **Sweet spot** — typically 300–500 tokens; always tune on your own data.

Rule of thumb

- 1 chunk $\approx$ 1 idea
- Tune size on real queries
- Measure retrieval, not vibes
- Re-chunk when accuracy stalls

Key takeaway: Tune chunk size empirically — it is the highest-leverage knob in RAG quality.

Hands-on

Goal: Build a document-ingestion foundation — load policy PDFs, clean, chunk and tag — ready for enterprise retrieval.

1 Load

Ingest policy PDFs with a document loader

2 Clean

Strip headers/footers, normalise text

3 Chunk

Recursive split, 400 tokens, 15% overlap

4 Tag

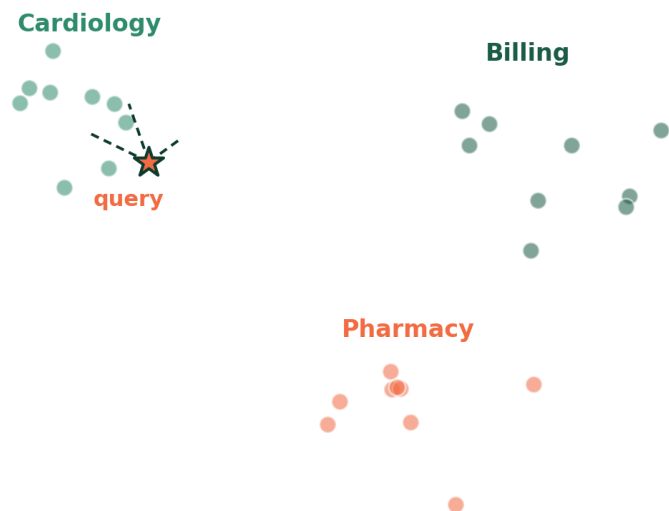
Add dept, effective-date, doc-type metadata

MODULE 7 Embeddings & Vector Search

embeddings • similarity search • vector databases • reranking • metadata filtering

Embeddings: meaning as coordinates

An embedding model maps text to a vector so that similar meaning lands nearby in space.



- **Cosine similarity** — angle between vectors ranks relevance.
- **Dimensions** — 384–1536 typical; more ≠ always better.
- **Domain matters** — medical text needs medical-aware models.
- **Same encoder** — index and query must share one model.

Similar meaning → nearby vectors (cosine similarity)

Key takeaway: Retrieval quality starts with the right embedding model for your domain.

Vector databases: the memory layer

- **Semantic search** — finds by meaning, not keywords.
- **ANN indexing** — millisecond retrieval across millions of vectors.
- **Hybrid search** — combine vector + keyword filters.
- **Governance** — host privately for data-residency compliance.
- **Live updates** — insert/delete as knowledge changes, no retrain.

FAISS

local, fast, library

Chroma

lightweight, dev-friendly

Qdrant / Milvus

production, scalable

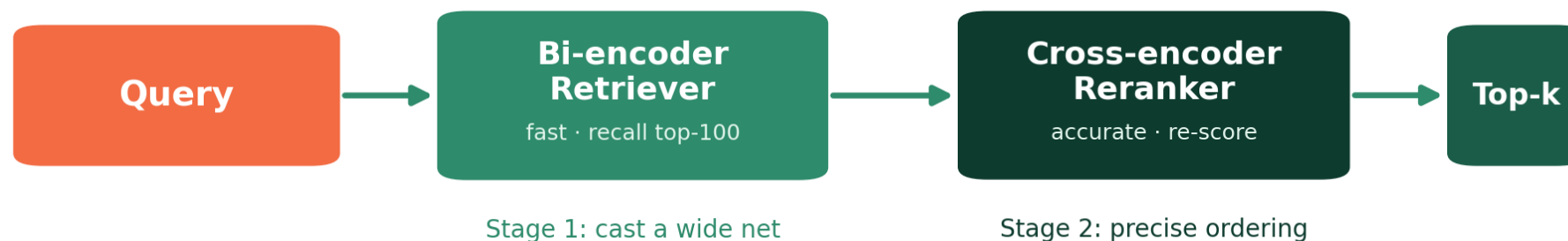
pgvector

inside Postgres

Key takeaway: Pick a store for your scale and governance needs — the API pattern is the same.

Reranking: recall first, then precision

A fast retriever casts a wide net; a slower cross-encoder re-scores the shortlist for accuracy.



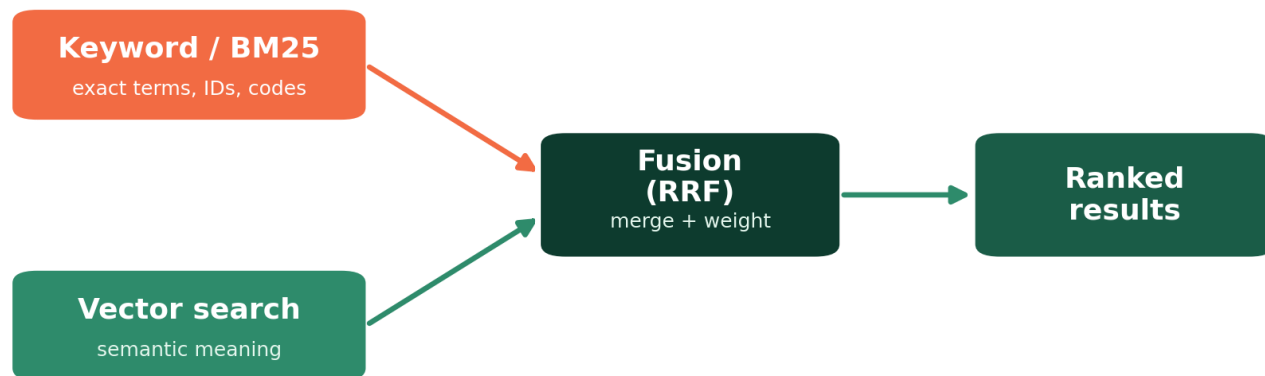
Metadata filtering: the governed layer

Tag chunks at ingestion, then filter BEFORE similarity search to enforce scope and access.



Hybrid retrieval: best of both worlds

Vectors miss exact codes and IDs; keyword search misses meaning. Fuse them.



- **BM25** — exact terms: claim IDs, ICD codes.
- **Vector** — paraphrases & synonyms.
- **RRF** — reciprocal-rank fusion merges both.

Hybrid = literal precision + semantic recall

Hands-on

Goal: Turn the tagged chunk store into a semantic search engine and measure retrieval precision.

1 Embed

Encode chunks with a domain model

2 Index

Load vectors into Qdrant / FAISS

3 Search

Top-k + metadata filters

4 Rerank

Cross-encoder to lift precision

MODULE 8 End-to-End Enterprise RAG

context assembly • prompt augmentation • multimodal • graph & audio RAG
• scaling limits

Context assembly & prompt augmentation

- **Assemble** — stitch retrieved chunks into a coherent context block.
- **Order matters** — most-relevant first; models weight early context more.
- **Augment** — inject context + question into a grounding prompt template.
- **Cite** — carry chunk IDs through so answers link back to sources.
- **Guard** — instruct the model to say “I don’t know” when context is thin.

Grounding prompt (shape)

Answer ONLY from the context.

Cite the source for each claim.

If missing, say you don’t know.

Context: {retrieved_chunks}

Question: {user_query}

Context-window management

The context window is the model's finite working memory — and it costs tokens.

- **Finite** — 8k–32k common; 128k–1M on frontier models.
- **Context explosion** — stuffing every chunk raises cost & latency, lowers accuracy.
- **Right-size** — retrieve fewer, better chunks; rerank then trim.
- **Compress** — summarise or dedupe context before the LLM call.

More context ≠ better

relevance beats volume

Latency

grows with tokens

Cost

you pay per token

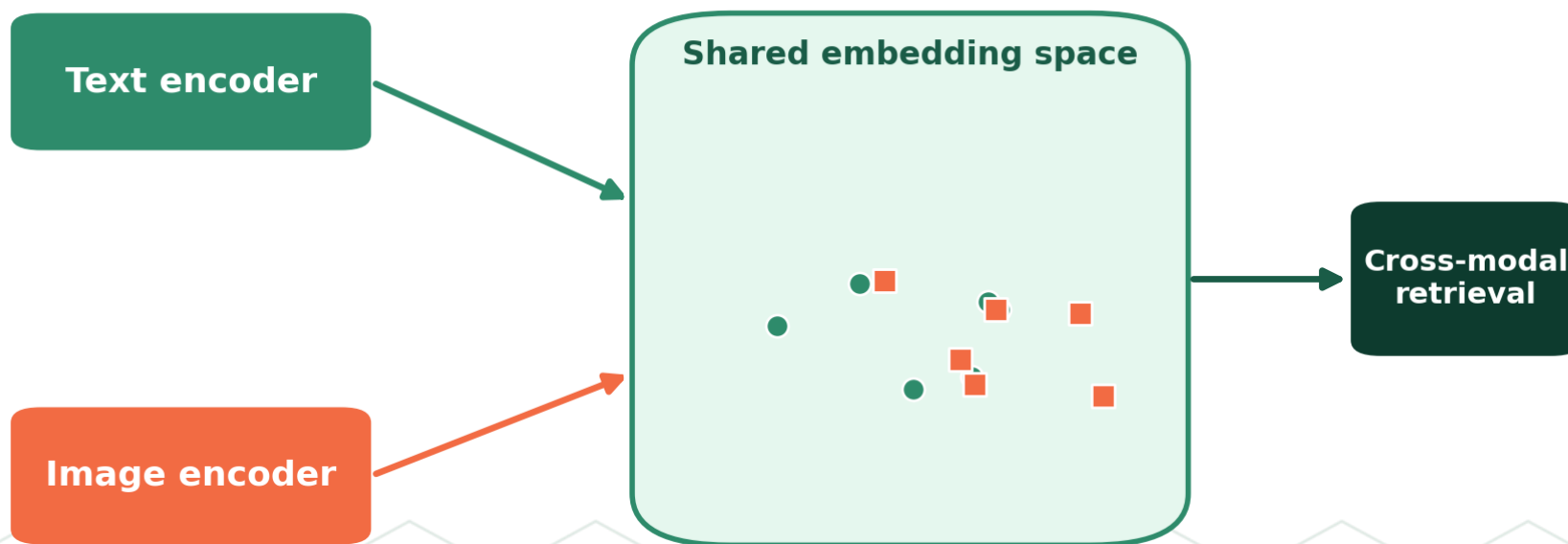
Real-world data: tables, PDFs & diagrams

- **Scanned PDFs** — need OCR + layout models (LayoutLM, DocFormer) before chunking.
- **Tables** — tokenizers flatten columns and lose relationships → parse to CSV / SQL or use hybrid lookup.
- **Images & diagrams** — caption with a vision-language model, or embed with CLIP.
- **Forms & charts** — extract both text and numeric metadata for retrieval.
- **Messy reality** — enterprise corpora mix all of the above — plan a pipeline, not a one-off.



Multimodal RAG

CLIP-style encoders place text and images in one shared space — query one, retrieve the other.



CLIP aligns text & images — query one, retrieve the other

Key takeaway: Multimodal retrieval unlocks scans, charts and medical imagery alongside text.

Graph RAG: connecting the dots

Traditional RAG returns isolated chunks; Graph RAG traverses entities and relationships.

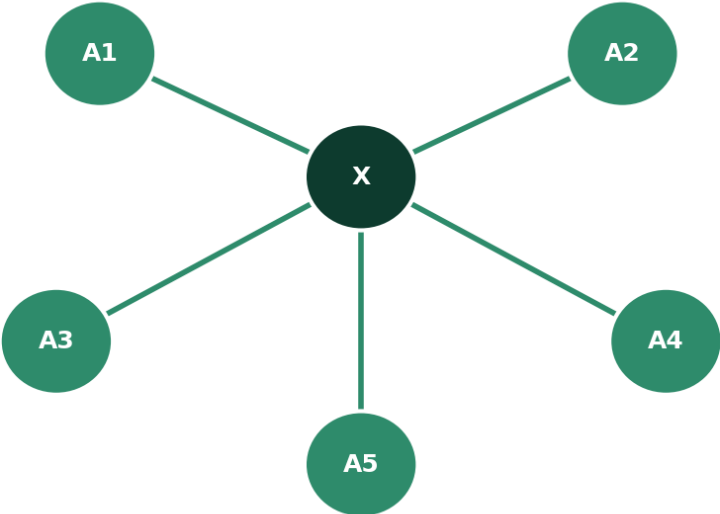
Traditional RAG



top-k isolated chunks

? LLM must guess how
chunks connect

Graph RAG

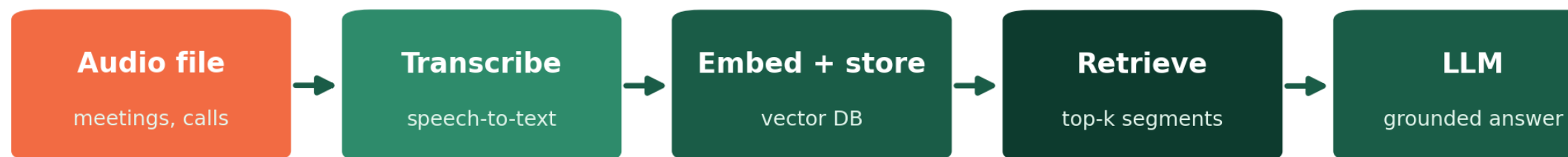


traverse entities + relationships
→ connected, complete answer

Key takeaway: Use Graph RAG when answers require reasoning across many connected facts.

Audio RAG

Plenty of enterprise knowledge is speech — calls, consults, dictations. Make it retrievable.



Speech becomes retrievable text — timestamps & speakers preserved as metadata

Key takeaway: Transcribe → embed → retrieve: speech joins the same RAG pipeline.

Why RAG fails at scale

- **Retrieval failures** — wrong or missing chunks → confident but wrong answers.
- **Context explosion** — too many chunks bury the signal and blow the budget.
- **Latency** — embedding + search + rerank + generation adds up under load.
- **Stale index** — knowledge drifts; yesterday's vectors answer today's question wrongly.
- **No evaluation** — without metrics, quality silently degrades.



Hands-on

1 Retrieve

Hybrid search + rerank

2 Assemble

Order & trim context

3 Augment

Grounding prompt + guardrails

4 Cite

Answer with source links

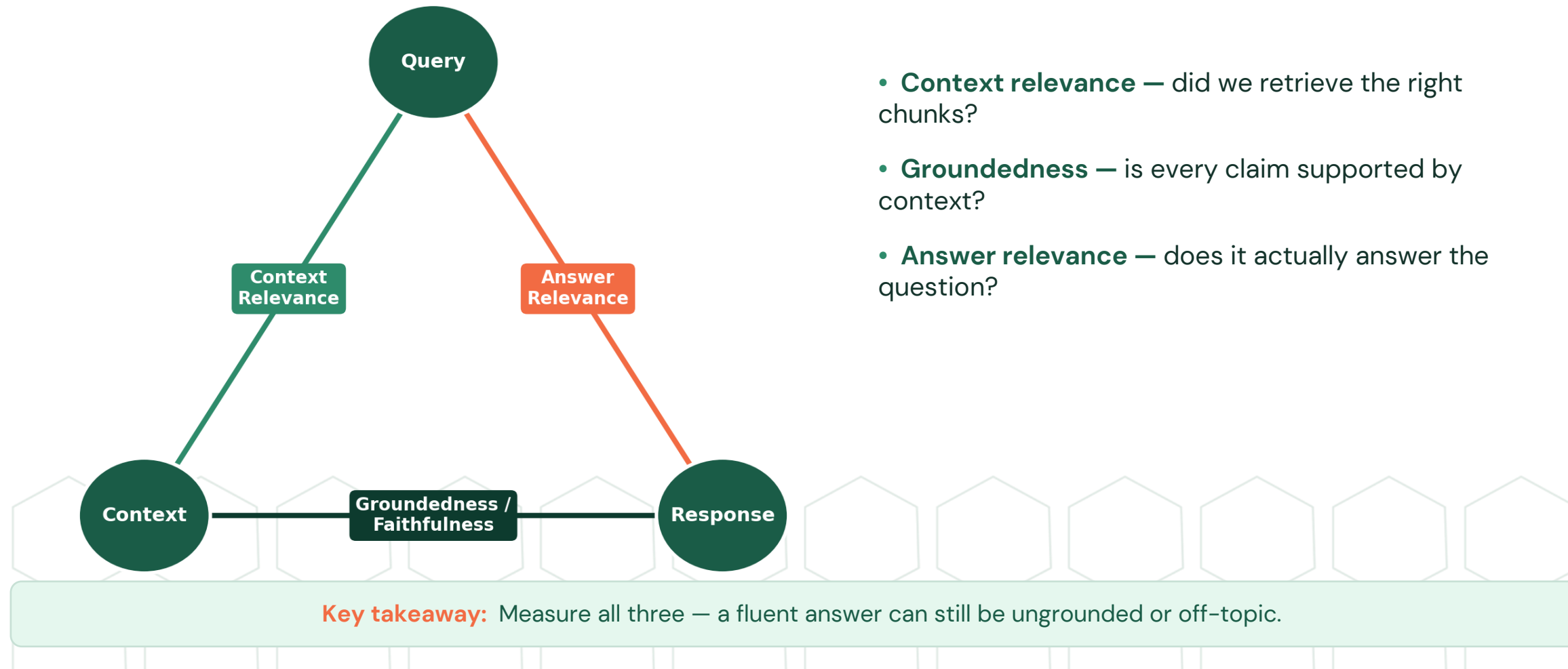


MODULE 9 Evaluation, Reliability & Governance

groundedness • faithfulness • hallucination detection • auditability •
responsible AI

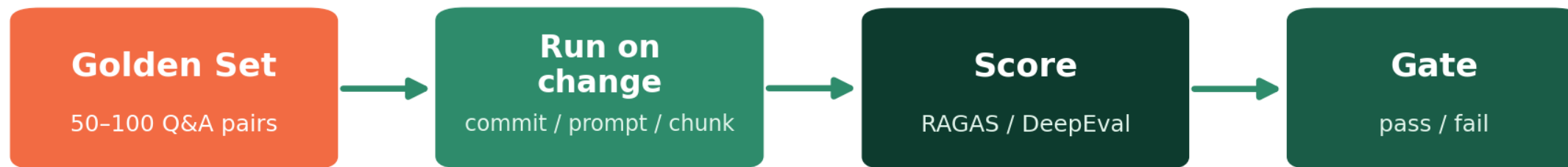
The RAG evaluation triad

Three relationships tell you whether a RAG answer can be trusted.



Golden sets & continuous evaluation

The “vibe check” is dead — curate a test set and run it automatically on every change.



Regression caught automatically — the “vibe check” is dead

Responsible AI & governance

- **Auditability** — log query, retrieved chunks, prompt and answer for every call.
- **PHI & privacy** — mask identifiers; enforce access control via metadata filters.
- **Hallucination controls** — require citations; abstain when context is insufficient.
- **Human-in-the-loop** — clinicians review before any patient-facing use.
- **Data residency** — host vectors privately to meet regulatory requirements.



Key takeaway: In healthcare, governance is not optional — it is the licence to deploy.

Hands-on — Evaluation Scorecard & Improvement Loop

Goal: Build a test set, score the assistant with RAGAS and close the improvement loop.

1 Curate

50–100 Q&A pairs from SMEs

2 Score

RAGAS: groundedness, relevance

3 Diagnose

Trace failures to a stage

4 Improve

Tune chunk/retrieve/prompt

Key takeaways

- **Foundations rule** — chunking + embeddings set the ceiling for everything else.
- **Retrieve smart** — hybrid search, reranking and metadata filters beat brute force.
- **Assemble carefully** — context order, trimming and grounding prompts drive trust.
- **Go beyond text** — multimodal, graph and audio RAG unlock real enterprise data.
- **Measure or regress** — golden sets, RAGAS and governance keep RAG reliable.



Thank you

Questions & discussion · InsightVerge AI Academy & Research Institute

